



Web Programming

Lecture 3

CSS

Mahhek Tahir



CSS - Cascade Style Sheet

Inline Styles

Style rules placed within an HTML element via the style attribute

```
<h1>Share Your Travels</h1>  
<h2 style="font-size: 24pt;">Description</h2>  
...  
<h2 style="font-size: 24pt; font-weight: bold;">Reviews</h2>
```

LISTING 3.1 Internal styles example

An inline style only affects the element it is defined within and will override any other style definitions for the properties used in the inline style.

Using inline styles is generally discouraged since they increase bandwidth and decrease maintainability.

Inline styles can however be handy for quickly testing out a style change.

Embedded Style Sheet

Style rules placed within the `<style>` element inside the `<head>` element

```
<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <style>
    h1 { font-size: 24pt; }
    h2 {
      font-size: 18pt;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Share Your Travels</h1>
  <h2>New York - Central Park</h2>
  ...

```

LISTING 3.2 Embedded styles example

While better than inline styles, using embedded styles is also by and large discouraged.

Since each HTML document has its own `<style>` element, it is more difficult to consistently style multiple documents when using embedded styles.

External Style Sheet

Style rules placed within an external text file with the .css extension

```
<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <link rel="stylesheet" href="styles.css" />
</head>
```

LISTING 3.3 Referencing an external style sheet

This is by far the most common place to locate style rules because it provides the best maintainability.

- When you make a change to an external style sheet, all HTML documents that reference that style sheet will automatically use the updated version.
- The browser is able to cache the external style sheet which can improve the performance of the site



CSS Selector

The CSS element Selector

The element selector selects HTML elements based on the element name.

Here, all <p> elements on the page will be center-aligned, with a red text color:

```
p {  
    text-align: center;  
    color: red;  
}
```



CSS Selector

The CSS element Selector

To select an element with a specific id, write a hash (#) character, followed by the id of the element.

The CSS rule below will be applied to the HTML element with id="para1":

```
#para1 {  
    text-align: center;  
    color: red;  
}
```



CSS Selector

HTML elements can also refer to more than one class.

```
<p class="center large">This paragraph refers  
to two classes.</p>
```

```
.center {  
    text-align: center;  
    color: red;  
}  
  
.large {  
    font-size: 100px;
```

```
    color: blue;
```




CSS Universal Selector

```
* {  
  text-align: center;  
  color: blue;  
}
```

CSS Grouping Selector

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}
```

Summary of Selectors



Selector Type	Syntax	Description	Example	Applies To
Universal	*	Selects all elements	<code>* { margin:0; }</code>	All elements in the document
Type / Element	<code>p, div, h1</code>	Select elements by tag name	<code>p { color: blue; }</code>	All <code><p></code> tags
Class	<code>.classname</code>	Select elements with a class	<code>.card { padding: 10px; }</code>	All elements with <code>class="card"</code>
ID	<code>#idname</code>	Select element with a specific id	<code>#header { text-align:center; }</code>	Single element with <code>id="header"</code>
Descendant	<code>parent child</code>	Select all elements inside a parent , at any depth	<code>div p { color:red; }</code>	All <code><p></code> inside <code><div></code> (nested or not) Anything inside the parent, at any depth.
Child	<code>parent > child</code>	Select direct children only	<code>.content > article { padding:10px; }</code>	Only <code><article></code> directly under <code>.content</code>
Attribute exact match	<code>[attr="value"]</code>	Select elements with an attribute equal to a value	<code>[data-type="news"] { background: yellow; }</code>	Only elements where attribute exactly matches
Attribute exists	<code>[attr]</code>	Select elements that have the attribute , any value	<code>[data-type] { border:1px solid black; }</code>	All elements that have <code>data-type</code>



Properties and Values - Text

```
<h1 style="background-color:DodgerBlue;color:white">Hello
```

```
World</h1>
```

Hello World

```
<p style="background-color:#FFFFFF;color:DodgerBlue;">Lorem  
ipsum...</p>
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

```
<h1 style="border:2px solid Tomato:">Hello World</h1>
```

Hello World

Properties and Values

```
body {  
  background-image: url("paper.gif");  
}
```

Hello World!

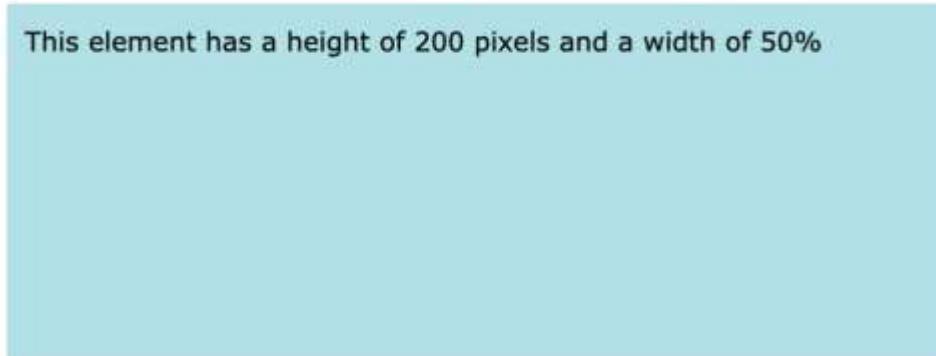
This page has an image as the background!



Height x Width

CSS height and width Examples

This element has a height of 200 pixels and a width of 50%



Example

Set the height and width of a <div> element:

```
div {  
  height: 200px;  
  width: 50%;  
  background-color: powderblue;  
}
```



Font

1. font-size: 40px; / 2.5em; / 100% / 24 pt;
2. font-family: Arial, "Times New Roman"
3. font-style: normal/italics/bold;
4. font-weight: normal/bold/900;

em is relative to parent div if parent is 24px
and child has 2em means child will be 48px

1pt = 1/72 inch (pt = print)



CSS Borders

```
p.dotted {border-style: dotted;}
p.dashed {border-style: dashed;}
p.solid {border-style: solid;}
p.double {border-style: double;}
p.groove {border-style: groove;}
p.ridge {border-style: ridge;}
p.inset {border-style: inset;}
p.outset {border-style: outset;}
p.none {border-style: none;}
p.hidden {border-style: hidden;}
p.mix {border-style: dotted
dashed solid double;}
```

A dotted border.

A dashed border.

A solid border.

A double border.

A groove border. The effect depends on the border-color value.

A ridge border. The effect depends on the border-color value.

An inset border. The effect depends on the border-color value.

An outset border. The effect depends on the border-color value.

No border.

A hidden border.

A mixed border.

CSS Borders

- ▶ `border-width: 5px;`
- ▶ `border-color: red;`

5px border-width

medium border-width

- ▶ `border-bottom: 1px solid #ccc;`
- ▶ `border-top: 1px solid #ccc !important;`
- ▶ `border-right: 1px solid #ccc;`
- ▶ `border-left: 1px solid #ccc !important;`



Align

float: left (default in english)/right

text-align: center/left/right

Different: "text-align" applies to the *content* of a box, while "float" applies to the box itself.



Margins and Paddings

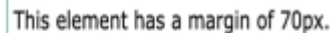
Margins are used to create space around elements, outside of any defined borders.

```
margin: 25px;
```

```
margin-top: 10px;
```

```
margin: 25px 50px 75px 100px;
```

- top margin is 25px - right margin is 50px - bottom margin is 75px - left margin is 100px



Padding is used to create space around an element's content, inside of any defined borders.

```
padding: 25px;
```

```
padding-top: 10px;
```

```
padding: 25px 50px 75px 100px;
```

- top padding is 25px - right padding is 50px - bottom padding is 75px - left padding is 100px





MARGIN - PADDING

```
div {  
    padding: 10px 20px;  
}
```

This means:

- Top = **10px**
- Bottom = **10px**
- Left = **20px**
- Right = **20px**

```
div {  
    padding: 10px 20px 30px;  
}
```

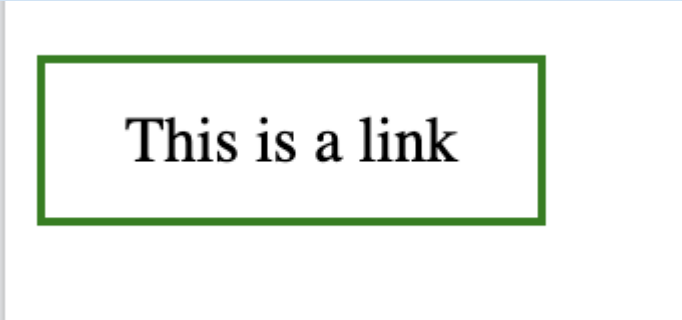
This means:

- Top = **10px**
- Left = **20px**
- Right = **20px**
- Bottom = **30px**

Anchor style

- `a:link` - a normal, unvisited link
- `a:visited` - a link the user has visited
- `a:hover` - a link when the user mouses over
- `a:active` - a link the moment it is clicked

```
a:link {  
  background-color: white;  
  color: black;  
  border: 2px solid green;  
  padding: 10px 20px;  
  text-align: center;  
  text-decoration: none;  
}
```



This is a link



Specificity

Specificity measure which css rule to apply when multiple css rules target the same styling property of an element, i.e.,

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      p {color: red;}

      #p_id {color: black;}

      .class{
        color: yellow;
      }
    </style>
  </head>
  <body>
    <p style="color: green" id="p_id" class="class">Hello World!</p>
  </body>
</html>
```



Specificity

The more specific the selector, the higher the priority. The order of priority is as follows (first one has the highest priority)

1. html style attribute
2. id
3. class
4. element selector



CSS Cascade

If two CSS rules have the same specificity, the browser determines which rule to apply using the following criteria:

1. **Internal vs. External CSS:** Rules defined in internal (embedded) CSS take precedence over rules in external CSS if they have the same specificity.
2. **Order of Appearance:** When two rules are in the same location (both in internal CSS or both in external CSS), the rule that appears **later** in the code will be applied.



Precedence

If the same css rule is being applied in more than one way, then the following order of precedence is followed:

1. inline css (html style attribute)
2. internal/embedded css
3. external css

Note: A css rule with !important always takes precedence.

EXAMPLE CODE:

```
background-color: red !important;
```


BOX MODEL



The **box model** defines the different parts of that shape, which are broken up into **four pieces**, or sub-boxes:

- **Content:** the inner content of your HTML element.
- **Padding:** a box that surrounds an HTML element's inner content. In the image below, it's the extra space around the content that also shares the background;



Here's a paragraph

- **Border:** a box that surrounds an HTML element's padding. In the image below, it's the darker space around the padding:



Here's a paragraph

- **Margin:** a functionally invisible box that surrounds an HTML element's border. The margin is meant to act as white space to separate HTML elements from each other. In the image below, it's the white spaces between and around the two paragraphs:



Here's a paragraph

Here's another paragraph



Display

- Primarily two types of display: block and inline
 - Block level elements are formatted visually as blocks / boxes (e.g. Paragraphs) – vertically stacked
 - Blocks may further contain other blocks or inline elements
 - Inline content is distributed in form of lines and does not introduce a new block content – horizontally aligned
 - `display:none` doesn't display element at all
- `display: inline;block;none`



Homework

HTML & CSS Test

Part 1: HTML

Create an HTML page that includes the following elements:

- A big header text with a title (e.g., "My Simple Web Page").
- A main section with at least one paragraph and an image.
- Another section that includes your name and the current year.

Part 2: CSS

Write a CSS stylesheet that styles your HTML page. Include the following styles:

- Use a font size and color for the header.
- Change the background color of the page.
- Style the main content section, giving the paragraph some padding and margin.
- Style the second section with a different background color.



Study



<https://www.w3schools.com/css/default.asp>